وثيقة المراجعة والتلخيص الأكاديمي

ASE · Chapter 5 · Software Design

شنو هذا: استخلصت **نمط أسئلة امتحانات حقيقية** من 19 ورقة امتحان (MRCAT/R18/R17) + 3 بنوك أسئلة. النمط تكرر بشكل واضح: **6 محاور بس**. **ليش مهم:** هذا اللي **يتوقع يجي بالامتحان فعلاً** — مو أسئلة الكتاب النظرية. **الصيغة:** سؤال بصيغة الامتحان + جواب نموذجي كامل + علامة تقديرية.

المحور 1 — Software Architecture (الأكثر تكراراً)

Q1.1 (14M) Define software architecture. Explain why it may be necessary to design the system architecture *before* writing detailed specifications. Compare function-oriented and object-oriented designs. **Answer:** Software architecture is the high-level structure of a system — the organisation of its components, their interrelationships, and the principles guiding their design and evolution. It must precede detailed design because the architecture **constrains and gives meaning to** every lower decision: module boundaries, interfaces and data structures cannot be chosen coherently until the overall structure is fixed, and changing the architecture late invalidates all the detailed work beneath it. **Function-oriented vs object-oriented:** function-oriented decomposes by *what the system does* (processes/functions → module hierarchy, documented with DFDs at analysis and structure charts at design); object-oriented decomposes by *the data/things the system manipulates* (objects and classes → interacting objects, documented with UML for both analysis and design). They are orthogonal views; FO suits algorithmic/process-dominated systems, OO suits entity-dominated systems and is more stable under requirement change.

Q1.2 (14M) Write the taxonomy of architectural styles and give a brief description of each. **Answer (model taxonomy):** - **Data-centred** — a central data store (repository/blackboard) accessed by independent components; e.g. a database-centred system. Good when many components share data; weak because the store is a single point of change. - **Data-flow** — components process data in a pipeline (batch sequential, pipes-and-filters). Good for transformation tasks; weak for interactive control. - **Call-and-return** — components call one another (main program/subprogram, object-oriented, layered). The mainstream style; layers restrict calls to the layer below. - **Hierarchical (virtual machine)** — an abstract machine layered over lower levels, e.g. an operating system or a layered interpreter. - **Independent components** — communicating processes or event systems (implicit invocation), where components react to events; good for extensible, decoupled systems. - **Repository/Blackboard** — a specialised data-centred style for problems with no deterministic solution strategy.

Q1.3 (7M) Distinguish between data flow architecture and call-and-return architecture. **Answer: Data-flow architecture** organises components around the movement of data — each component transforms input into output and passes it on (pipes-and-filters, batch-sequential); control is implicit in the data flow. **Call-and-return architecture** organises components around explicit invocation — a component calls another and waits for its return (main/subprogram, object-oriented, layered); control is explicit in the call structure. Data-flow suits transformation pipelines; call-and-return suits interactive and object-oriented systems.

المحور 2 — Design Concepts / Principles

Q2.1 (14M) Explain the fundamental concepts/principles of software design: Abstraction, Modularity, Information hiding, Refactoring. **Answer:** - **Abstraction** — eliminate the irrelevant and amplify the essentials. Two forms: *procedural abstraction* (a named procedure whose body is hidden) and *data abstraction* (a named data type whose representation is hidden). Abstraction lets a designer reason at one level without the details below. - **Modularity** — decompose the system into separately named, manageable units with well-defined interfaces. It is the single attribute that makes a program intellectually manageable, trading module development effort against integration effort (cost curve with a minimum region). - **Information hiding (encapsulation)** — separate the external aspect of an entity from its internal implementation. It is the mechanism that makes modules depend only on stable interfaces, and thus the direct enabler of low coupling. - **Refactoring** — the disciplined restructuring of existing code/design to improve its internal structure **without changing external behaviour**, used to hold design quality as the system evolves.

Q2.2 (7M) Describe the characteristics and criteria for a good design. **Answer:** A good design must satisfy **four criteria** (slide 8): (i) **correct and complete** — every requirement is satisfied; (ii) **understandable** — structured so it can be read and reasoned about by both coders and testers; (iii) **at the right level of abstraction** — not over- or under-detailed; (iv) **maintainable** — it can absorb change locally. The book adds the classical architectural attributes given by Vitruvius — **durability, utility, and charm** (3 attributes, *not* 4) — to describe what a good design *is*: it must weather change, serve its purpose, and present a coherent structure. Operationally, quality is achieved by the design principle: **high cohesion and low coupling** in every module. A design is assessed against these criteria by review before implementation.

Q2.3 (14M) Explain a two-level process of design. Why should system design be finished *before* detailed design, rather than starting detailed design right after the requirements specification? **Answer:** Design has two levels: **architectural (high-level/system) design**, which partitions the system into major components and defines their relationships and interfaces, and **detailed design**, which specifies the internal logic, data structures and algorithms of each component. The architecture must be finished first because it is the **context** that gives detailed design its meaning: the module boundaries, the interfaces, and the distribution of responsibility all come from the architecture, and any detailed design done before the architecture is fixed would have to be redone. Starting detailed design immediately after the SRS produces the classic failure of the flowchart era — a sudden, unvalidated jump from specification to low-level decisions, which is a major source of errors.

Q2.4 (7M) Discuss the golden rules of user interface design. **Answer:** Three golden rules: (1) **Place the user in control** — the interface must let the user drive (allow undo, flexible interaction, no forced sequences, adapt to the user's skill). (2) **Reduce the user's memory load** — minimise what the user must remember (meaningful defaults, progressive disclosure, visible context, consistent metaphors). (3) **Make the interface consistent** — consistent across the system, across the platform, and across the interaction patterns the user has already learned.

المحور 3 — Cohesion & Coupling (الأكثر جوهرية)

Q3.1 (14M) Define coupling and cohesion. Explain the types of each and how they are useful in arriving at a good design, with an example. **Answer: Coupling** = degree of interdependence between modules; **cohesion** = degree to which the elements within a module are functionally related. *Coupling best → worst:* Data → Stamp →

Control → External → Common → Content. *Cohesion best → worst:* Functional → Sequential → Communicational → Procedural → Temporal → Logical → Coincidental. Their usefulness: a good design maximises cohesion and minimises coupling, producing **functionally independent** modules that give error isolation, reuse potential, and understandability. **Example:** passing only a studentID between edit_record and retrieve_record is data coupling (good); passing the whole student record plus an EOF flag is tight coupling with tramp data (bad).

Q3.2 (7M) What is modularity? Write its importance by keeping maximization of cohesion and minimization of coupling as the basis. **Answer:** Modularity is the decomposition of a system into well-defined, manageable units with well-defined interfaces, and it is the single attribute that makes a program intellectually manageable. Its importance is realised through the cohesion/coupling principle: by **maximising cohesion** each module performs a single well-defined task and can be understood, tested and reused in isolation; by **minimising coupling** the ripple effect of change is localised and error propagation is contained. Together they produce functional independence — the direct enabler of design clarity, easier implementation, debugging, testing, documentation and maintenance.

Q3.3 (7M) If a module has logical cohesion, what kind of coupling is it likely to have with others? Explain the ripple effect and how coupling affects program stability. **Answer:** Logical cohesion (a module grouping functions of the same *logical class*, chosen at run time) typically brings **control coupling** — a flag/parameter must tell the module which logical function to run — and often **common coupling**, because such utility modules tend to touch global data. **Ripple effect:** a change in one module forces changes in the modules that depend on it; the higher the coupling and the more hidden the dependency, the more severe the ripple. Program **stability** is therefore inversely related to coupling: data coupling confines a change to the interface, while content coupling lets a change propagate silently through internals with no compiler warning. High coupling among modules makes the design hard to understand and maintain and increases development effort.

Object-Oriented Design Process — 4 المحور

Q4.1 (14M) Explain the object-oriented design process using UML, with suitable examples. **Answer (the SIX-step process, per Fig 25 — verified against both slide 72 and book printed p.232):** 1. **Create the use case model** — identify actors, write use cases, draw the use-case diagram. 2. **Draw the activity diagram** (if required) — model the flow of control from activity to activity, showing the dynamic nature of the system. 3. **Draw the interaction diagram** — sequence/collaboration diagrams showing messages between objects over time. 4. **Draw the class diagram** — the static structure: classes, attributes, operations, and relationships (association, aggregation, dependency, generalisation). 5. **Draw the state chart / object diagram** (if required) — the states of an object and the events that cause transitions. 6. **Draw the component and deployment diagrams** → produce the design documents — the physical/modular aspects and their distribution. Worked example: the Library Management System (issue-book sequence diagram, class diagram of entity classes).

Q4.2 (7M) What are the merits of the object-oriented design process? Discuss the reasons for improvement using OOD. **Answer:** OOD begins from the real-world things of the problem (objects characterised by attributes and behaviour), so the model is **natural and stable**: requirements changes tend to add or alter operations on entities rather than replace entities. **Encapsulation** localises change to the affected class; **inheritance** promotes reuse of common structure and behaviour; **polymorphism** lets new behaviour be added by subclassing rather

than rewriting; and the design is **independent of any implementation language**. The result is easier maintenance, extension and understanding, and better handling of change.

Q4.3 (7M) Explain: Objects, Message, Abstraction, Class, Inheritance, Polymorphism, and the relationship between ADTs and classes. **Answer: Object** — an entity that retains state and offers operations to examine/affect it. **Message** — a request from one object to another (target identity + operation name + parameters), often implemented as a procedure/function call. **Abstraction** — eliminating the irrelevant and amplifying the essentials. **Class** — a set of objects sharing a common structure (attributes) and behaviour (operations); it acts as a blueprint. **Inheritance** — a subclass/derived class acquires state and behaviour from a superclass/base class. **Polymorphism** — abstracting just the interface of an operation and leaving the implementation to subclasses. **ADT vs class:** an ADT *specifies* data plus operations with hidden representation; a class *implements* that specification, binds data and operations, and can instantiate objects.

Q4.4 (7M) Can we have inheritance without polymorphism? Explain. **Answer:** Yes. Inheritance is the *mechanism* by which a subclass acquires structure and behaviour from a superclass. Polymorphism is the *ability* to treat different subclasses through a common interface with different implementations. A subclass can inherit attributes and operations without ever overriding any operation polymorphically, so inheritance can exist without polymorphism. (The converse is also common in practice — but it is inheritance that is the enabling mechanism.)

المحور 5 — Data Design

Q5.1 (7M) Write a short note on data design, and list the guidelines for data design. **Answer:** Data design transforms the information domain model into the data structures required for implementation — it refines the data objects and relationships identified during analysis into the structures (files, tables, databases, internal data stores) that the software will use. Guidelines: systematically analyse the data objects and their relationships; identify all data structures and the operations on them; establish a data dictionary describing each data item; keep data structures simple and hide their representation (data abstraction); design the data structures at the same time as the program structure (they influence each other); and choose a representation matched to the dominant operations, considering access frequency, storage cost and integrity.

المحور 6 — Assessment & Design Quality

Q6.1 (7M) How can we assess the quality of a software design? **Answer:** A software design is assessed against the design criteria: **correctness/completeness** (does it satisfy all requirements?), **understandability** (can it be read and reasoned about?), **level of abstraction** (is it at the right level?), and **maintainability** (can it absorb change locally?). Operationally the design is evaluated by measuring its **coupling and cohesion** (aim: low coupling, high cohesion → functional independence), by reviewing it against the SDD organisation (are the decomposition, dependency, interface and detail views all present and consistent?), and by **design reviews/walkthroughs** carried out before implementation, since design errors caught after coding are the most expensive to fix.

Q6.2 (7M) Provide examples of three data abstractions and the procedural abstractions that manipulate them. **Answer:** (1) **Stack** — data abstraction: push/pop/peek operations hiding the array/pointer representation; procedural abstraction: the push routine. (2) **Date** — data abstraction: a date type with set/get-day/month/year

and comparison operations hiding the day-count representation; procedural abstraction: the date-comparison routine. (3) **Polygon** — data abstraction: a shape type with draw/move/rotate hiding the vertex list; procedural abstraction: the draw routine. In each case the *data abstraction* names the thing and hides its representation, while the *procedural abstraction* names the operation and hides its algorithm.

خريطة الأولوية (شنو يذاكر أول)

الأولوية	المحور	التكرار بالامتحانات
★★★	Coupling & Cohesion	يظهر بكل ورقة تقريباً
★★★	OOD process + merits	8+ ورقات
☆★★	Software architecture + styles	7+ ورقات
☆★★	Design concepts/principles	6+ ورقات
☆☆★	UI golden rules	5 ورقات
☆☆★	Data design	4 ورقات

مصدر النمط: 19 ورقة امتحان MRCET (R17/R18) + بنوك NRCMEC و GitHub و MAIT. كل جواب مُتحقق منه من الكتاب/السلايدات.

Retrieval set (active recall)

السؤال	الجواب
كم محور امتحاني متكرر؟	6
أثقل محور بالامتحان؟	Software Architecture
أكثر محور جوهرية؟	Coupling & Cohesion
مصدر الأنماط؟	19 ورقة امتحان (MRCAT/R18/R17) + 3 بنوك أسئلة
شنو الأولوية للمذاكرة؟	Coupling/Cohesion ثم OOD process ثم Architecture ثم UI